

多态与模版 (OOP)

黄民烈

aihuang@tsinghua.edu.cn

<https://coai.cs.tsinghua.edu.cn/html>

课程团队：刘知远 任炬 黄民烈

上期要点回顾

- 向上类型转换
- 对象切片
- 函数调用捆绑
- 虚函数和虚函数表
- 虚函数和构造函数、析构函数
- 重写覆盖，**override**和**final**

本讲内容提要

- 纯虚函数与抽象类
- 向下类型转换
- 多重继承中的虚函数
- 多态
- 函数模板与类模板

纯虚函数

- 虚函数还可以进一步声明为纯虚函数（如下所示），包含纯虚函数的类，通常被称为“**抽象类**”。

virtual 返回类型 函数名(形式参数) = 0;

- 抽象类不允许定义对象，定义基类为抽象类的主要用途是为派生类规定**共性“接口”**

```
class A {
```

```
public:
```

```
    virtual void f() = 0; ///可在类外定义函数体提供默认实现。派生类通过 A::f() 调用  
};
```

A obj; ///**不准抽象类定义对象！编译不通过！**

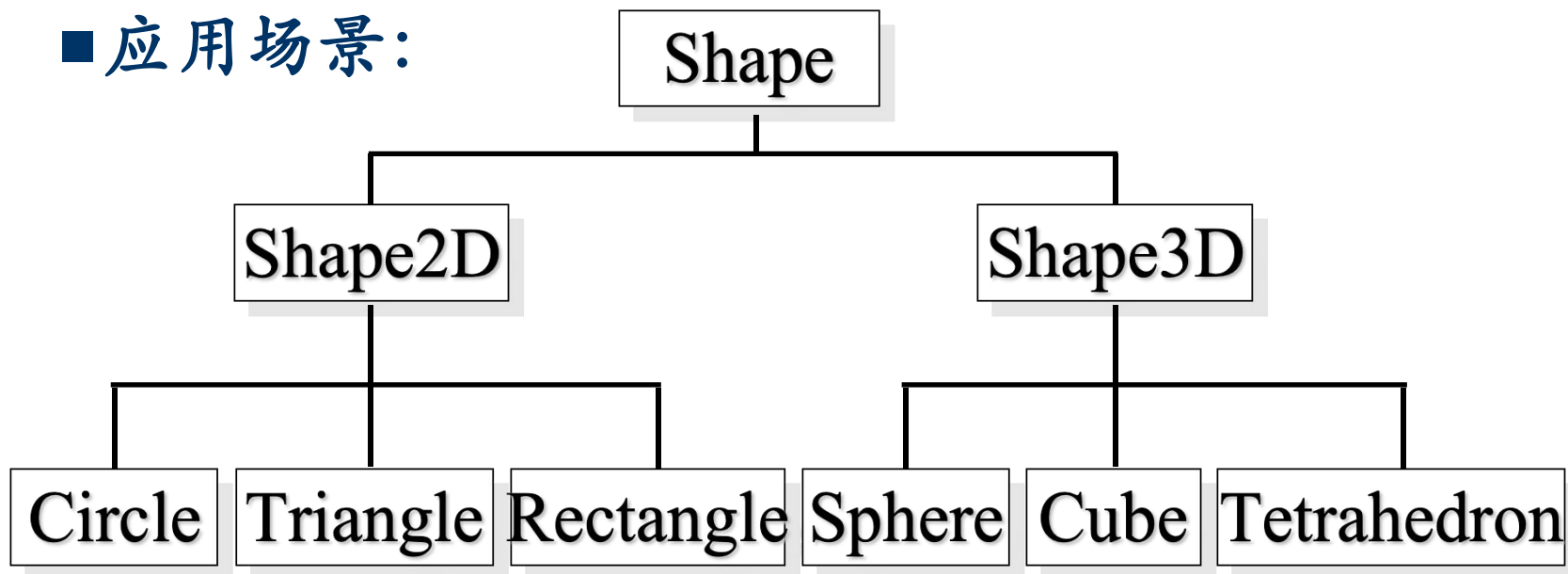
抽象类

■ 定义：含有至少一个纯虚函数。

■ 特点：

- 不允许定义对象。
- 只能为派生类提供接口。
- 能避免对象切片：保证只有指针和引用能被向上类型转换。

■ 应用场景：



纯虚函数与抽象类示例

```
#include <iostream>
using namespace std;

class Pet {
public:
    virtual void motion()=0;
};

void Pet::motion(){ cout << "Pet motion: " << endl; }

class Dog: public Pet {
public:
    void motion() override {Pet::motion(); cout << "dog run" << endl; }
};

class Bird: public Pet {
public:
    void motion() override {Pet::motion(); cout << "bird fly" << endl; }
};

int main() {
    Pet* p = new Dog; /// 向上类型转换
    p->motion();
    p = new Bird; /// 向上类型转换
    p->motion();
    //p = new Pet; /// 不允许定义抽象类对象
    return 0;
}
```

运行结果

```
Pet motion:
dog run
Pet motion:
bird fly
```

抽象类

- 基类纯虚函数被派生类重写覆盖之前仍是纯虚函数。因此当继承一个抽象类时，除纯虚析构函数外（后面解释），必须实现所有纯虚函数，否则继承出的类也是抽象类。

抽象类示例

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void func()=0;
};

class Derive1: public Base {}; //Derive1仍为抽象类
class Derive2: public Base {
public:
    void func() {
        cout << "Derive2::func" << endl;
    }
};

int main()
{
    // Derive1 d1; //编译错误, Derive1仍为抽象类
    Derive2 d2;
    d2.func();
    return 0;
}
```

运行结果

Derive2::func

纯虚析构函数

■回顾：虚函数与析构函数

- 析构函数能是虚的，且常常是虚的。虚析构函数仍需定义函数体。
- 虚析构函数的用途：当删除基类对象指针时，编译器将根据指针所指对象的实际类型，调用相应的析构函数。

纯虚析构函数

■ 析构函数也可以是纯虚函数

- 纯虚析构函数**仍然需要函数体**
- 目的：使基类成为**抽象类**，不能创建基类的对象。如果有其他函数是纯虚函数，则析构函数无论是否为纯虚的，基类均为抽象类。

```
class Base { public: virtual ~Base()=0; };
```

```
Base::~~Base() {} /// 必须有函数体
```

```
class Derive : public Base {};
```

```
int main() {
```

```
    Base b; /// 编译错误，基类是抽象类
```

```
    Derive d1; /// 派生类不必实现纯虚析构函数
```

```
    return 0;
```

```
}
```

纯虚析构函数

■ 纯虚析构函数和一般纯虚函数

- 一般的纯虚函数被派生类重写覆盖之前仍是纯虚函数。如果派生类不覆盖纯虚函数，那么派生类也是抽象类。
- 纯虚析构函数除外
- 对于纯虚析构函数而言，即便派生类中不显式实现，编译器也会自动合成默认析构函数。因此，即使派生类不显式覆盖纯虚析构函数，只要派生类覆盖了其他纯虚函数，该派生类就不是抽象类，可以定义派生类对象。

纯虚析构函数 示例

```
#include <iostream>
using namespace std;

class Base{
public:
    virtual ~Base()=0;
};
Base::~~Base() {cout<<"Base destroyed"<<endl;}

class Derive1: public Base {};
class Derive2: public Base {
public:
    virtual ~Derive2() {cout<<"Derive2 destroyed"<<endl;} };

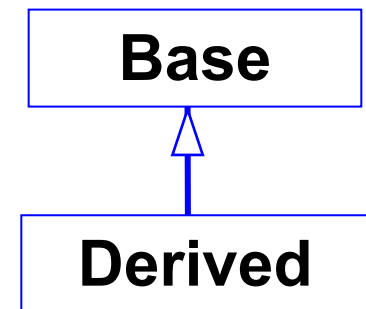
int main()
{
    Base* p1 = new Derive1;
    Base* p2 = new Derive2;
    delete p1;
    cout << "-----" << endl;
    delete p2;
    return 0;
}
```

运行结果

```
Base destroyed
-----
Derive2 destroyed
Base destroyed
```

回顾：向上类型转换

- 派生类对象/引用/指针转换成基类对象/引用/指针，称为**向上类型转换**。只对**public**继承有效，在继承图上是上升的；对**private**、**protected**继承无效。
- 向上类型转换（派生类到基类）可以由编译器**自动完成**，是一种**隐式**类型转换。
- **凡是**接受基类对象/引用/指针的地方（如函数参数），**都可以**使用派生类对象/引用/指针，编译器会自动将派生类对象转换为基类对象以便使用。



向下类型转换

- 基类指针/引用转换成派生类指针/引用，则称为**向下类型转换**。（类层次中向下移动）
- 为什么要向下类型转换？
 - 当我们用基类指针表示各种派生类时(向上类型转换)，保留了他们的**共性**，但是丢失了他们的**特性**。如果此时要表现特性，则可以使用向下类型转换。
 - 比如我们可以使用**基类指针数组**对各种派生类对象进行管理，当具体处理时我们可以将基类指针转换为实际的派生类指针，进而调用派生类**专有**的接口。
- 如何确保转换的正确性？
 - 如何保证基类指针指向的对象也可以被要转换的派生类的指针指向？—— 借助虚函数表进行**动态类型检查**！

向下类型转换

■ C++ 提供了一个特殊的显式类型转换，称为 **dynamic_cast**，是一种安全的向下类型转换。

- 使用 dynamic_cast 的对象必须有虚函数，因为它使用了存储在虚函数表中的信息判断实际的类型。

■ 使用方法：

- obj_p, obj_r 分别是 T1 类型的指针和引用
- `T2* pObj = dynamic_cast<T2*>(obj_p);`
// 转换为 T2 指针，运行时失败返回 **nullptr**
- `T2& refObj = dynamic_cast<T2&>(obj_r);`
// 转换为 T2 引用，运行时失败抛出 **bad_cast** 异常
- 在向下转换中，T1 必须是多态类型（声明或继承了至少一个虚函数的类），否则不过编译

向下类型转换

- 如果我们知道正在处理的是哪些类型，可以使用 **static_cast** 来避免这种开销。

- static_cast 在 **编译** 时静态浏览类层次，只检查 **继承关系**。没有继承关系的类之间，必须具有转换途径才能进行转换（要么自定义，要么是语言语法支持），否则不过编译。运行时无法确认是否正确转换。

- **static_cast** 使用方法：

- obj_p, obj_r 分别是 T1 类型的指针和引用
- T2* pObj = **static_cast**<T2*>(obj_p);
// 转换为 T2 指针
- T2& refObj = **static_cast**<T2&>(obj_r);
// 转换为 T2 引用
- 不安全：不保证指向目标是 T2 对象，可能导致非法内存访问。

示例

```
#include <iostream>
using namespace std;
class B { public: virtual void f() {} };
class D : public B { public: int i{2018}; };

int main() {
    D d; B b;
    // D d1 = static_cast<D>(b); ///未定义类型转换方式
    // D d2 = dynamic_cast<D>(b); ///只允许指针和引用转换

    D* pd1 = static_cast<D*>(&b); /// 有继承关系，允许转换
    if (pd1 != nullptr){
        cout << "static_cast, B*(B) --> D*: OK" << endl;
        cout << "D::i=" << pd1->i << endl;
    } /// 但是不安全：对D中成员i可能非法访问

    D* pd2 = dynamic_cast<D*>(&b);
    if (pd2 == nullptr) /// 不允许不安全的转换
        cout << "dynamic_cast, B*(B) --> D*: FAILED" << endl;

    return 0;
}
```

```
static_cast, B*(B) --> D*:OK
D::i=124455624
dynamic_cast, B*(B) --> D*: FAILED
```

示例

```
#include <iostream>
using namespace std;
class B { public: virtual void f() {} };
class D : public B { public: int i{2018}; };

int main() {
    D d; B b;
    // D d1 = static_cast<D>(b); ///未定义类型转换
    // D d2 = dynamic_cast<D>(b); ///只允许指针和引用转换

    B* pb = &d;
    D* pd3 = static_cast<D*>(pb);
    if (pd3 != nullptr){
        cout << "static_cast, B*(D) --> D*: OK" << endl;
        cout << "D::i=" << pd3->i << endl;}
    D* pd4 = dynamic_cast<D*>(pb);
    if (pd4 != nullptr){/// 转换正确
        cout << "dynamic_cast, B*(D) --> D*: OK" << endl;
        cout << "D::i=" << pd4->i << endl;
    }
    return 0;
}
```

```
static_cast, B*(D) --> D*: OK
D::i=2018
dynamic_cast, B*(D) --> D*: OK
D::i=2018
```

向下类型转换

dynamic_cast与static_cast

■ 相同点：

- 都可完成向下类型转换。

■ 不同点：

- static_cast在编译时静态执行向下类型转换。
- dynamic_cast会在运行时检查被转换的对象是否确实是正确的派生类。额外的检查需要 RTTI (Run-Time Type Information)，因此要比static_cast慢一些，但是更安全。

■ 一般使用dynamic_cast进行向下类型转换

向下类型转换

重要原则(清楚指针所指向的真正对象):

- 1) 指针或引用的向上转换总是安全的;
- 2) 向下转换时用**dynamic_cast**, 安全检查;
- 3) 避免对象之间的转换。

类型转换其他用法

dynamic_cast与static_cast

- **dynamic_cast**也能对指针或引用进行向上类型转换。(较少使用，因为向上转换支持隐式转换)
- **static_cast**也能对不同对象类型进行转换

参考：

https://en.cppreference.com/w/cpp/language/dynamic_cast

https://en.cppreference.com/w/cpp/language/static_cast

向上向下类型转换与虚函数表

■ 对于基类中有虚函数的情况：

■ 向上类型转换：

- 转换为基类**指针或引用**，则对应虚函数表仍为派生类的虚函数表（晚绑定）。
- 转换为基类**对象**，产生对象切片，调用基类函数（早绑定）。

■ 向下类型转换：

- `dynamic_cast`通过虚函数表来判断是否能进行向下类型转换。

示例

```
#include <iostream>
using namespace std;

class Pet { public: virtual ~Pet() {} };
class Dog : public Pet {
public: void run() { cout << "dog run" << endl; }
};
class Bird : public Pet {
public: void fly() { cout << "bird fly" << endl; }
};

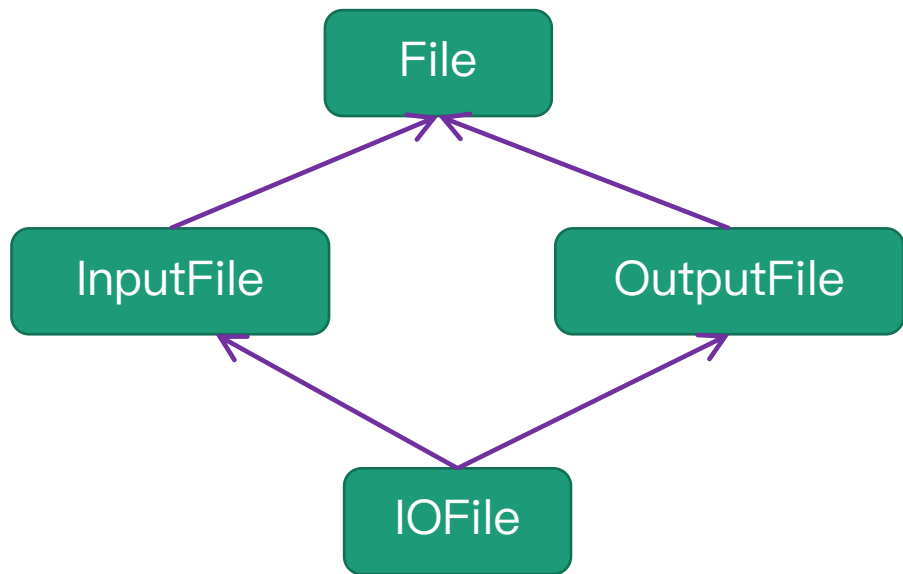
void action(Pet* p) {
    auto d = dynamic_cast<Dog*>(p); /// 向下类型转换
    auto b = dynamic_cast<Bird*>(p); /// 向下类型转换
    if (d) /// 运行时根据实际类型表现特性
        d->run();
    else if(b)
        b->fly();
}

int main() {
    Pet* p[2];
    p[0] = new Dog; /// 向上类型转换
    p[1] = new Bird; /// 向上类型转换
    for (int i = 0; i < 2; ++i) {
        action(p[i]);
    }
    return 0;
}
```

运行结果

```
dog run
bird fly
```

回忆：多重继承



■ 利：

- 清晰，符合直觉
- 结合多个接口

■ 弊：

- 二义性：如果派生类D继承的两个基类A,B，有同名成员a，则访问D中a时，编译器无法判断要访问的哪一个基类成员。
- 钻石型继承树（DOD: Diamond Of Death）带来的数据冗余：右图中如果 InputFile 和 OutputFile 都含有继承自 File 的 filename 变量，则 IOFile 会有两份独立的 filename，而这实际上并不需要。

多重继承中的虚函数

■ Best Practice:

- 最多继承一个非抽象类 (is-a)
- 可以继承多个抽象类 (接口)

■ 为什么?

- 避免 多重继承的二义性
- 利用 一个对象可以实现多个接口

多重继承示例

```
#include <iostream>
using namespace std;

class WhatCanSpeak {
public:
    virtual ~WhatCanSpeak() {}
    virtual void speak() = 0; };

class WhatCanMotion {
public:
    virtual ~WhatCanMotion() {}
    virtual void motion() = 0; };

class Human : public WhatCanSpeak, public WhatCanMotion
{
    void speak() { cout << "say" << endl; }
    void motion() { cout << "walk" << endl; }
};

void doSpeak(WhatCanSpeak* obj) { obj->speak(); }
void doMotion(WhatCanMotion* obj) { obj->motion(); }

int main()
{
    Human human;
    doSpeak(&human); doMotion(&human);
    return 0;
}
```

运行结果

say
walk

多态 (Polymorphism)

- 按照**基类**的接口定义，调用**指针或引用**所指对象的接口函数，函数执行过程因对象**实际**所属**派生类**的不同而呈现不同的效果（表现），这个现象被称为“多态”。
- 当利用**基类指针/引用**调用函数时
 - 虚函数在**运行**时确定执行哪个版本，取决于引用或指针对象的真实类型
 - 非虚函数在**编译**时绑定
- 当利用**类的对象**直接调用函数时
 - 无论什么函数，均在**编译**时绑定
- 产生多态效果的条件：**继承 && 虚函数 && (引用 或 指针)**

多态 (Polymorphism)

- 多态，使得C++语言可以用一段相同的代码，在运行时完成不同的任务，这些不同运行结果的差异由派生类之间的差异决定。
- 好处：
 - 通过基类定好接口后，不必对每一个派生类特殊处理，只需要调用抽象基类的接口即可。大大提高程序的可复用性。
 - 不同派生类对同一接口的实现不同，能达到不同的效果，提高了程序可拓展性和可维护性。

多态示例

```
#include <iostream>
using namespace std;
```

```
class Animal{
public:
    void action() {
        speak();
        motion();
    }
    virtual void speak() { cout << "Animal speak" << endl; }
    virtual void motion() { cout << "Animal motion" << endl; }
};
```



复用基类接口

```
class Bird : public Animal
{
public:
    void speak() { cout << "Bird singing" << endl; }
    void motion() { cout << "Bird flying" << endl; }
};
```

多态示例

```
class Fish : public Animal
{
public:
    void speak() { cout << "Fish cannot speak ..." << endl; }
    void motion() { cout << "Fish swimming" << endl; }
};
```

```
int main() {
    Fish fish;
    Bird bird;
    fish.action();    ///不同调用方法
    bird.action();

    Animal *pBase1 = new Fish;
    Animal *pBase2 = new Bird;
    pBase1->action(); ///同一调用方法，根据
    pBase2->action(); ///实际类型完成相应动作
    return 0;
}
```

运行结果

```
Fish cannot speak ...
Fish swimming
Bird singing
Bird flying
Fish cannot speak ...
Fish swimming
Bird singing
Bird flying
```

多态 (Polymorphism)

- 应用：TEMPLATE METHOD设计模式
 - 在接口的一个方法中定义算法的骨架
 - 将一些步骤的实现延迟到子类中
 - 使得子类可以在不改变算法结构的情况下，重新定义算法中的某些步骤。
- 模板方法是一种源代码重用的基本技术，在类库的设计实现中应用十分广泛，因为这个设计模式能有效地解决“类库提供公共行为”与“用户定制特殊细节”之间的折中平衡。

Template设计模式

```
#include <iostream>
using namespace std;
```

```
class Base{
public:
    void action() {
        step1();
        step2();
        step3();
    }
    virtual void step1() { cout << "Base::step1" << endl; }
    virtual void step2() { cout << "Base::step2" << endl; }
    virtual void step3() { cout << "Base::step3" << endl; }
};

class Derived1 : public Base{
    void step1() { cout << "Derived1::step1" << endl; }
};
```



算法骨架

Template设计模式

```
class Derived2 : public Base{  
    void step2() { cout << "Derived2::step2" << endl; }  
};
```

```
int main(){  
    Base* ba[] = {new Base, new Derived1, new Derived2};  
    for (int i = 0; i < 3; ++i) {  
        ba[i]->action();  
        cout<<"==="<<endl;  
    }  
    return 0;  
}
```

运行结果

```
Base::step1  
Base::step2  
Base::step3  
===  
Derived1::step1  
Base::step2  
Base::step3  
===  
Base::step1  
Derived2::step2  
Base::step3  
===
```

函数模板和类模板

继承与组合提供了重用对象代码的方法，
而C++的模板特征提供了重用源代码的方法。

模板：引入

- 实现一个整数排序算法接口：

void sort(int *data, int len);

- 实现一个浮点数排序算法接口：

void sort(float *data, int len);

- 实现一个自定义类型排序算法接口：

void sort(myClass *data, int len);

- 明明实现逻辑是一样的，为什么要写多遍？

函数模板

- 有些算法实现与类型无关，所以可以将函数的参数类型也定义为一种特殊的“参数”，这样就得到了“函数模板”。

- 定义函数模板的方法

```
template <typename T> ReturnType Func(Args);
```

- 如：任意类型两个变量相加的“函数模板”

```
template <typename T>  
T sum(T a, T b) { return a + b; }
```

- 注：**typename**也可换为**class**

```
template <class T>  
T sum(T a, T b) { return a + b; }
```

函数模板

- 函数模板在调用时，编译器能自动推导出实际参数的类型（这个过程叫做**实例化**）。
- 所以，形式上调用一个函数模板与普通函数没有区别，如
 - `cout << sum(9, 3);`
 - `cout << sum(2.1, 5.7);`
- **调用类型需要满足函数的要求**。本例中，要求类型 `T` 定义了加法运算符。
- 当多个参数的类型不一致时，无法推导：
`cout << sum(9, 2.1);` **// 编译错误**
- 可以手工指定调用类型：`sum<int>(9, 2.1)`

函数模板示例

```
#include <iostream>
#include <algorithm>
```

```
template<class T>
void sort(T* data, int len)
{
    for(int i = 0; i < len; i++){ //选择排序
        for(int j = i + 1; j < len; j++) {
            if(data[i] > data[j])
                std::swap(data[i], data[j]); //交换元素位置
        }
    }
}
```

```
template<class T>
void output(T* data, int len)
{
    for(int i = 0; i < len; i++)
        std::cout << data[i] << " ";
    std::cout << std::endl;
}
```

函数模板示例

```
int main()
{
    int arr_a[] = {3,2,4,1,5};
    sort(arr_a, 5); //调用int类型的sort
    output(arr_a, 5); //调用int类型的output

    float arr_b[] = {3.2, 2.1, 4.3, 1.5, 5.7};
    sort(arr_b, 5); //调用float类型的sort
    output(arr_b, 5); //调用float类型的output
    return 0;
}
```

运行结果

1 2 3 4 5

1.5 2.1 3.2 4.3 5.7

函数模板示例

```
class MyInt
{
public:
    int data;
    MyInt(int val): data(val) {};
};

int main()
{
    MyInt arr_c[] = {3, 2, 4, 1, 5};
    sort(arr_c, 5);
    output(arr_c, 5);
    return 0;
}
```

模板也可以支持
自定义类型

编译错误?

模板的编译错误一般会引起大量报错(几百行), 我们只看最上方几行

main.cpp: In instantiation of 'void sort(T*, int) [with T = MyInt]':

main.cpp:33:15: required from here

main.cpp:9:15: error: no match for 'operator>' (operand types are 'MyInt' and 'MyInt')

问题: sort中需要operator> 但MyInt并不支持

函数模板示例

```
class MyInt
{
public:
    int data;
    MyInt(int val): data(val) {};
    bool operator>(const MyInt& b){ //用于sort
        return data > b.data;
    }
    friend std::ostream& //用于output
        operator<<(std::ostream& out, const MyInt& obj){
        out << obj.data;
        return out;
    }
};

int main()
{
    MyInt arr_c[] = {3, 2, 4, 1, 5};
    sort(arr_c, 5);
    output(arr_c, 5);
    return 0;
}
```

运行结果

1 2 3 4 5

模板原理

- 对模板的处理是在**编译期**进行的，每当编译器发现对模板的一种参数的使用，就生成对应参数的一份代码。

```
template<typename T>  
T sum(T a, T b) {return a + b;}
```

```
int main() {  
    int a = sum(1, 2); //生成并编译int sum(int, int)  
    double b = sum(1.0, 2.0);  
        //生成并编译double sum(double, double)  
    return 0;  
}
```

- 也带来了问题：模板库必须在头文件中实现，不可以分开编译（**请思考为什么？**）

为什么声明和定义要在一起

① x.o 没有**具体类型调用**，不生成任何代码

x.h ←————→ x.cpp

```
template<typename T>
T sum(T a, T b);
```

```
template<typename T>
T sum(T a, T b) {return a + b;}
```

② main.o 只看到声明，
看不到定义，无法实例化

③ main.o 和 x.o 链接时，只能找到 **sum**
的引用，但在 **x.o** 中找不到 **sum** 的定义，
链接失败 (undefined reference)

```
#include x.h
int main() {
    int a = sum(1, 2); //生成并编译int sum(int, int)
    double b = sum(1.0, 2.0);
                //生成并编译double sum(double, double)
    return 0;
}
```

main.cpp

类模板

- 在定义类时也可以将一些类型信息抽取出来，用模板参数来替换，从而使类更具通用性。这种类被称为“类模板”。例如：

```
#include <iostream>
using namespace std;
```

```
template <typename T> class A {
    T data;
public:
    A(T _data): data(_data) {}
    void print() { cout << data << endl; }
};
int main() {
    A<int> a(1);
    a.print();
    return 0;
}
```

类模板

■ 类模板中成员函数的类外定义

```
#include <iostream>
using namespace std;
```

```
template <typename T> class A {
    T data;
public:
    A(T _data): data(_data) {}
    void print();
};
```

```
template<typename T>
void A<T>::print() { cout << data << endl; }
int main() {
    A<int> a(1);
    a.print();
    return 0;
}
```

类模板

■ 类模板的“模板参数”

- 类型参数：使用typename或class标记
- 非类型参数：**整数**，枚举，指针（指向对象或函数），引用（引用对象或引用函数）。无符号整数(unsigned)比较常用。如：

```
template<typename T, unsigned size>  
class array {  
    T elems[size];  
};
```

```
array<char, 10> array0;
```

类模板

- 所有模板参数必须在编译期确定，不可以使用变量。

```
template<typename T, unsigned size>
class array {
    T elems[size];
};

int main(){
    int n = 5;
    //array<char, n> array0; //不能使用变量
    const int m = 5;
    array<char, m> array1; //可以使用常量
    array<char, 5> array2; //或具体数值
    return 0;
}
```

类模板示例

```
#include <iostream>
#include <algorithm>
```

```
template<class T, unsigned size>
class MyArr
{
    T data[size];
public:
    void sort(){
        for(int i = 0; i < size; i++){ //选择排序
            for(int j = i + 1; j < size; j++) {
                if(data[i] > data[j])
                    std::swap(data[i], data[j]); //交换两者位置
            }
        }
    }
    void output(){
        for(int i = 0; i < size; i++)
            std::cout << data[i] << " ";
        std::cout << std::endl;
    }
    .....
}
```


类模板示例

```
void input(){
    for(int i = 0; i < size; i++)
        std::cin >> data[i];
}

};

int main()
{
    MyArr<int, 5> arr_a;
    arr_a.input();
    arr_a.sort();
    arr_a.output();

    MyArr<float, 5> arr_b;
    arr_b.input();
    arr_b.sort();
    arr_b.output();
    return 0;
}
```

输入

3 2 4 1 5

3.2 2.1 4.3 1.5 5.7

运行结果

1 2 3 4 5

1.5 2.1 3.2 4.3 5.7

模板与多态

- 模板使用泛型标记，使用 **同一段代码**，来关联不同但相似的特定行为，最后可以获得不同的结果。模板也是 **多态** 的一种体现。
- 但模板的关联是在编译期处理，称为 **静多态**。
 - 往往和函数重载同时使用
 - 高效，省去函数调用
 - 编译后代码增多
- 基于继承和虚函数的多态在运行期处理，称为 **动多态**
 - 运行时，灵活方便
 - 侵入式，必须继承
 - 存在函数调用

OOP核心思想

- OOP的核心思想是数据抽象、继承与动态绑定
- 数据抽象：类的接口与实现分离
- 继承：建立相关类型的层次关系（基类与派生类）
- 动态绑定：统一使用基类指针，实现多态行为

OOP核心思想

■ OOP的核心思想是数据抽象、继承与动态绑定

■ 数据抽象：类的接口与实现分离

- 回顾Animal/模板设计的例子

■ 继承：建立相关类型的层次关系（基类与派生类）

- Is-a、is-implementing-in-terms-of: 客观世界的认知关系

■ 动态绑定：统一使用基类指针，实现多态行为

- 虚函数
- 类型转换，模板

课后阅读

■ 《C++编程思想》

- 多态性与虚函数，第十五章
- 模板，第十六章

成员函数模板 (自学)

- 普通类的成员函数，也可以定义为模板函数，如：

```
class normal_class {  
public:  
    int value;  
    template<typename T> void set(T const& v) {  
        value = int(v);  
    }    /// 在类内定义  
    template<typename T> T get();  
};  
template<typename T>    /// 在类外定义  
T normal_class::get() {  
    return T(value);  
}
```

成员函数模板 (自学)

- 模板类的成员函数，也可有额外的模板参数

```
template<typename T0> class A {  
    T0 value;  
public:  
    template<typename T1> void set(T1 const& v){  
        value = T0(v); /// 将T1转换为T0储存  
    }          /// 在类内定义  
    template<typename T1> T1 get();  
};  
template<typename T0> template<typename T1>  
T1 A<T0>::get(){ return T1(value);}  
    /// 类外定义， 将T0转换为T1返回
```

成员函数模板 (自学)

- 注意不能写成:

```
template<typename T0, typename T1>
```

```
T1 A<T0>::get(){ return T1(value);}
```

/// 错误, 与多个参数的模板混淆

```
template<typename T0> template<typename T1>
```

```
T1 A<T0>::get(){ return T1(value);} /// 正确
```


成员函数模板 (自学)

■ 多个参数的模板

多个参数的类模板:

```
template<typename T0, typename T1> class A  
{  
    ...  
};
```

多个参数的函数模板

```
template<typename T0, typename T1> void func(T0  
a1, T1 a2) {...}
```

成员函数模板 (自学)

- 模板使用中通常可以从参数自动推导类型，无法推导时需要指定
- 普通类模板的成员函数，也可有额外的模板参数

```
template<typename T0> class A {  
    T0 value;  
public:  
    template<typename T1> void set(T1 const& v){  
        value = T0(v); /// 将T1转换为T0储存  
    } /// 在类内定义  
    template<typename T1> T1 get();  
};  
template<typename T0> template<typename T1>  
T1 A<T0>::get(){ return T1(value);} /// 类外定义，将T0转换为T1返回  
int main() {  
    A<int> a;  
    a.set(5); /// 自动推导5为整数类型  
    double t = a.get<double>(); /// 手动指定返回值类型  
    return 0;  
}
```

课后练习 1

- 我们在学函数重载的时候知道，编译器只通过函数名和参数列表来区分不同的函数，因此重载的函数和原函数不能只有返回值不同。那么在重写和虚函数的情况下呢？
- 现在有以下代码，试试看能不能编译通过，如果不行，则修改它使得能编译通过。运行编译后的程序，看看结果是否符合你的预期。尝试解释在重载、重写、虚函数重写的情况下 **C++** 为什么要这样设计。

```
#include <iostream>
```

```
class Base {
```

```
public:
```

```
    virtual void f() {std::cout << "Call Base void f() " << std::endl;}
```

```
    virtual void g() {std::cout << "Call Base void g() " << std::endl;}
```

```
    void h() {std::cout << "Call Base void h()" << std::endl;}
```

```
};
```

```
class Derived : public Base {
```

```
public:
```

```
    void f() { std::cout << "Call Derive void f() " << std::endl; }
```

```
    int g() {
```

```
        std::cout << "Call Derive int g()" << std::endl;
```

```
        return 0;
```

```
    }
```

```
    int h() {
```

```
        std::cout << "Call Base int h()" << std::endl;
```

```
        return 0;
```

```
    }
```

```
};
```

课后练习 2

■ 仿照 C++ 的 **vector** 实现一个 **Vector** 类，要求使用模板以支持任意类型的元素，并且至少具有以下成员函数（省略了函数参数），测试代码见下页

- `void push_back();` // 在尾部插入一个元素
- `void pop_back();` // 将尾部最后一个参数弹出
- `int size();` // 返回 vector 的大小
- `operator[]();` // 重载 `[]` 运算符

```

#include <iostream>

#include "Vector.h"

#include <string>

int main() {
    Vector<int> int_v;
    Vector<std::string> string_v;
    Vector<Vector<int>> matrix;
    for (int i = 0; i < 100; ++i) int_v.push_back(i);
    for (int i = 0; i < 10; ++i) string_v.push_back("abcd");
    for (int i = 0; i < 50; ++i) {
        Vector<int> tmp_v;
        for (int j = 0; j < 50; ++j) tmp_v.append(j);
        matrix.append(tmp_v);
    }
    std::cout << int_v.size() << std::endl;
    for (int i = 0; i < 100; i += 2) std::cout << int_v[i] << " ";
    std::cout << std::endl;
    while (string_v.size() > 0) {
        std::cout << string_v[string_v.size() - 1] << std::endl;
        string_v.pop_back();
    }
    return 0;
}

```

结束